

UNIVERSIDADE PAULISTA

Brian Pereira Ventura – RA - N095DF1

Giulia Ferrari – RA - G783HF2

Joaquim José Mendes de Souza – RA - N079657

Lucas Inocêncio de França - RA - N239239

Rodrigo Pamplona Soares – RA - N2409H8

Washington Santos Souza Filho – RA - N046FD9

ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS

**DESENVOLVIMENTO DE UMA FERRAMENTA
PARA COMUNICAÇÃO EM REDE**

SÃO JOSÉ DOS CAMPOS

2025

Brian Pereira Ventura – RA - N095DF1

Giulia Ferrari – RA - G783HF2

Joaquim José Mendes de Souza – RA - N079657

Lucas Inocência de França - RA - N239239

Rodrigo Pamplona Soares – RA - N2409H8

Washington Santos Souza Filho – RA - N046FD9

ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS

DESENVOLVIMENTO DE UMA FERRAMENTA PARA COMUNICAÇÃO EM REDE

Atividades Práticas Supervisionadas do
4º e 5º Semestres do Curso de Ciência
da Computação da **Universidade
Paulista – UNIP.**

Coordenador: Prof. Fernando A. **Gotti**
Prof Responsável: André Yoshimi
Kusumoto

**SÃO JOSÉ DOS CAMPOS
2025**

SUMÁRIO

1 - INTRODUÇÃO	1
2 - FUNDAMENTOS DA COMUNICAÇÃO DE DADOS EM REDE.....	2
2.1 - ELEMENTOS ESSENCIAIS DA COMUNICAÇÃO	2
2.2 - MEIOS FÍSICOS DE TRANSMISSÃO EM REDES DE COMPUTADORES...	2
2.2.1 - CABO DE PAR TRANÇADO.....	3
2.2.2 - FIBRA ÓPTICA	3
2.2.3 - TRANSMISSÃO SEM FIOS	3
2.3 - MODELO OSI	4
2.4 - CLASSIFICAÇÃO DAS REDES DE COMPUTADORES	6
3 - PROTOCOLO TCP/IP.....	6
3.1 - CAMADAS DO TCP/IP	7
3.2 - PROTOCOLO IP (IPV4 E IPV6)	8
3.2 - IPV4 X IPV6.....	8
3.3 - ENDEREÇOS PÚBLICOS E PRIVADOS	9
3.4 - NAT (NETWORK ADDRESS TRANSLATION) E ESCASSEZ DE ENDEREÇOS IP	10
3.5 - IPV6.....	11
4 - SOCKETS DE BERKELEY	12
4.1 - CONCEITO	12
4.2 - FUNDAMENTOS DOS SOCKETS	13
4.2.1 - ENDEREÇAMENTO	13
4.3 - TIPOS DE SOCKETS.....	13
4.4 - GERENCIAMENTO DE MÚLTIPLAS CONEXÕES	15
4.5 - THREADS OU PROCESSOS.....	15
4.6 - MULTIPLEXAÇÃO COM SELECT(), POLL() OU EPOLL()	15

4.6.1 - SELECT()	15
4.6.2 - POLL()	16
4.6.3- EPOLL()	16
4.7 - ASSÍNCRONO / NÃO BLOQUEANTE (ASYNC I/O)	16
5 - PLANO DE DESENVOLVIMENTO DA APLICAÇÃO	17
6 - RELATÓRIOS COM LINHAS DE CÓDIGO.....	20
6.1 - CÓDIGO DO CLIENTE.....	20
6.2 - CÓDIGO DO SERVIDOR	25

1 - INTRODUÇÃO

A comunicação em rede é um dos pilares da tecnologia moderna, sendo essencial para a troca de informações entre sistemas distribuídos. Ela permite que dados sejam transmitidos de forma rápida, segura e eficiente, possibilitando o funcionamento de aplicações que fazem parte do nosso cotidiano. Este trabalho tem como objetivo desenvolver uma ferramenta de comunicação em rede, utilizando o modelo cliente-servidor e oferecendo suporte a múltiplas conexões simultâneas.

Ao longo do projeto, são explorados conceitos teóricos e práticos da área, com destaque para os protocolos de comunicação, os meios físicos de transmissão e o uso de *sockets* como interface de programação. A aplicação desenvolvida inclui ainda mecanismos de segurança por meio da criptografia, garantindo a proteção dos dados transmitidos entre os sistemas envolvidos.

A metodologia adotada envolve pesquisa bibliográfica, análise comparativa de diferentes protocolos de rede e o desenvolvimento de um *software* em Python. Para isso, são utilizadas bibliotecas específicas tanto para a comunicação em rede quanto para a criação de interface gráfica, proporcionando uma experiência mais completa ao usuário.

Por fim, este trabalho busca contribuir para o entendimento dos desafios e soluções no desenvolvimento de aplicações de rede, servindo como base para futuros estudos e projetos na área de redes de computadores.

2 - FUNDAMENTOS DA COMUNICAÇÃO DE DADOS EM REDE

A comunicação de dados constitui a base tecnológica que possibilita o intercâmbio de informações na era digital. Este processo complexo envolve a transmissão estruturada de dados entre sistemas computacionais através de diversos meios físicos e lógicos. A eficácia dessa comunicação depende fundamentalmente da implementação adequada de protocolos padronizados, que estabelecem as regras para formatação, transmissão, recepção e interpretação dos dados. Entre os protocolos mais relevantes destacam-se o conjunto TCP/IP, fundamental para operação da Internet, e tecnologias sem fio como *Bluetooth* e *Wi-Fi*, cada um com suas características específicas de operação e aplicação. (Tanenbaum & Wetherall, 2011).

2.1 - ELEMENTOS ESSENCIAIS DA COMUNICAÇÃO

O processo de comunicação digital compreende cinco componentes fundamentais que atuam de forma integrada. O elemento transmissor (emissor) é responsável pela origem e envio da informação, podendo ser qualquer dispositivo computacional com capacidade de processamento e transmissão de dados. O receptor, por sua vez, executa a função inversa, captando e interpretando as mensagens transmitidas. A conexão entre esses elementos é estabelecida através do meio de comunicação, que pode ser físico (cabos metálicos, fibras ópticas) ou lógico (ondas eletromagnéticas em redes sem fio). A mensagem representa o conteúdo informacional propriamente dito, enquanto os protocolos funcionam como o conjunto de normas e procedimentos que regulamentam todo o processo comunicacional, garantindo a interoperabilidade entre sistemas heterogêneos. (Tanenbaum & Wetherall, 2011).

2.2 - MEIOS FÍSICOS DE TRANSMISSÃO EM REDES DE COMPUTADORES

Os meios físicos de transmissão são fundamentais para a comunicação de dispositivos em redes, sendo responsáveis por transportar os sinais entre eles. Conforme Tanenbaum e Wetherall (2011), a escolha do meio de transmissão influencia diretamente vários parâmetros, como velocidade, confiabilidade e alcance da comunicação. Vamos abordar os três meios mais utilizados hoje: o cabo de par trançado, muito utilizado em redes locais; a fibra óptica, conhecida por sua elevada capacidade de transmissão e velocidade; e a comunicação sem fio, que proporciona mobilidade. Cada uma dessas soluções apresenta diferentes características, técnicas distintas que as tornam adequadas para aplicações específicas.

2.2.1 - CABO DE PAR TRANÇADO

O cabo de par trançado, tecnicamente denominado *Twisted Pair*, consiste em pares de condutores de cobre entrelaçados, trançados em forma helicoidal para minimizar interferências eletromagnéticas. Essa técnica se divide em duas categorias principais, o UTP (*Unshielded Twisted Pair*), que apresenta um custo menor e mais flexibilidade por não possuir blindagem adicional, sendo adequado para ambientes com baixa interferência; e o STP (*Shielded Twisted Pair*), que incorpora uma proteção metálica para reduzir ruídos eletromagnéticos, sendo particularmente indicado para instalações industriais ou áreas com elevado índice de interferência. As principais vantagens desta tecnologia são sua economia, facilidade de implementação e ampla compatibilidade com os padrões *Ethernet* convencionais. Mas em contrapartida, apresenta limitações significativas em termos de largura de banda quando comparamos à fibra óptica, além de demonstrar sensibilidade a interferências em transmissões de longo alcance, onde ocorre atenuação progressiva do sinal. (Tanenbaum & Wetherall, 2011)

2.2.2 - FIBRA ÓPTICA

A fibra óptica representa um marco tecnológico na transmissão de dados, utilizando pulsos luminosos para transportar informações com velocidades extremamente elevadas e completa imunidade a interferências eletromagnéticas. Esta solução apresenta duas variantes principais: a fibra multimodo, que permite a propagação de múltiplos feixes luminosos simultaneamente, sendo mais acessível financeiramente, mas com alcance limitado a aproximadamente 2 quilômetros; e a fibra monomodo, que opera com um único feixe de luz, possibilitando transmissões em distâncias superiores a 100 quilômetros com largura de banda significativamente maior. As principais vantagens desta tecnologia são sua capacidade de transmissão na ordem de *terabit* por segundo, atenuação extremamente reduzida do sinal e segurança intrínseca, já que não emite radiação eletromagnética passível de interceptação. Em contrapartida, os custos de implantação e manutenção são consideravelmente mais altos, e sua natureza frágil exige procedimentos especiais durante a instalação. (Tanenbaum & Wetherall, 2011)

2.2.3 - TRANSMISSÃO SEM FIOS

A comunicação sem fio revolucionou o conceito de conectividade, eliminando a necessidade de infraestrutura física através da utilização de ondas eletromagnéticas.

Entre as implementações mais relevantes destacam-se o padrão *Wi-Fi* para redes locais, a tecnologia *Bluetooth* para conexões de curto alcance e as redes celulares de quarta e quinta gerações (4G/5G) para cobertura em larga escala. A principal vantagem desta abordagem é a mobilidade irrestrita que proporciona, além de uma significativa redução de custos com infraestrutura física. No entanto, o desempenho está sujeito a interferências ambientais e obstruções físicas, exigindo protocolos avançados de criptografia para garantir a segurança das transmissões. Outro desafio inerente é o compartilhamento do espectro radioelétrico, que pode resultar em congestionamento em cenários de alta densidade de usuários. (Tanenbaum & Wetherall, 2011)

2.3 - MODELO OSI

Segundo Forouzan, Fegan e Griesi (2008), o modelo de referência OSI (*Open Systems Interconnection*), criado pela Organização Internacional de Normalização (ISO), é um modelo conceitual que padroniza a comunicação entre sistemas distintos por meio de protocolos em sete camadas:

Camada Física: Responsável pela transmissão física dos dados, incluindo *hardware* como cabos e *switches*. Nesta camada, os dados são convertidos em sinais binários (bits) para transmissão.

Camada de Enlace de Dados: Atua na comunicação entre dispositivos na mesma rede, dividindo os pacotes da camada de rede em quadros menores. Também gerencia o controle de fluxo e a detecção/correção de erros. (FOROUZAN; FEGAN; GRIESI, 2008)

Camada de Rede: Responsável pelo roteamento de dados entre redes diferentes. Se dois dispositivos estão na mesma rede, essa camada torna-se desnecessária. Ela determina o caminho mais eficiente para os dados alcançarem seu destino.

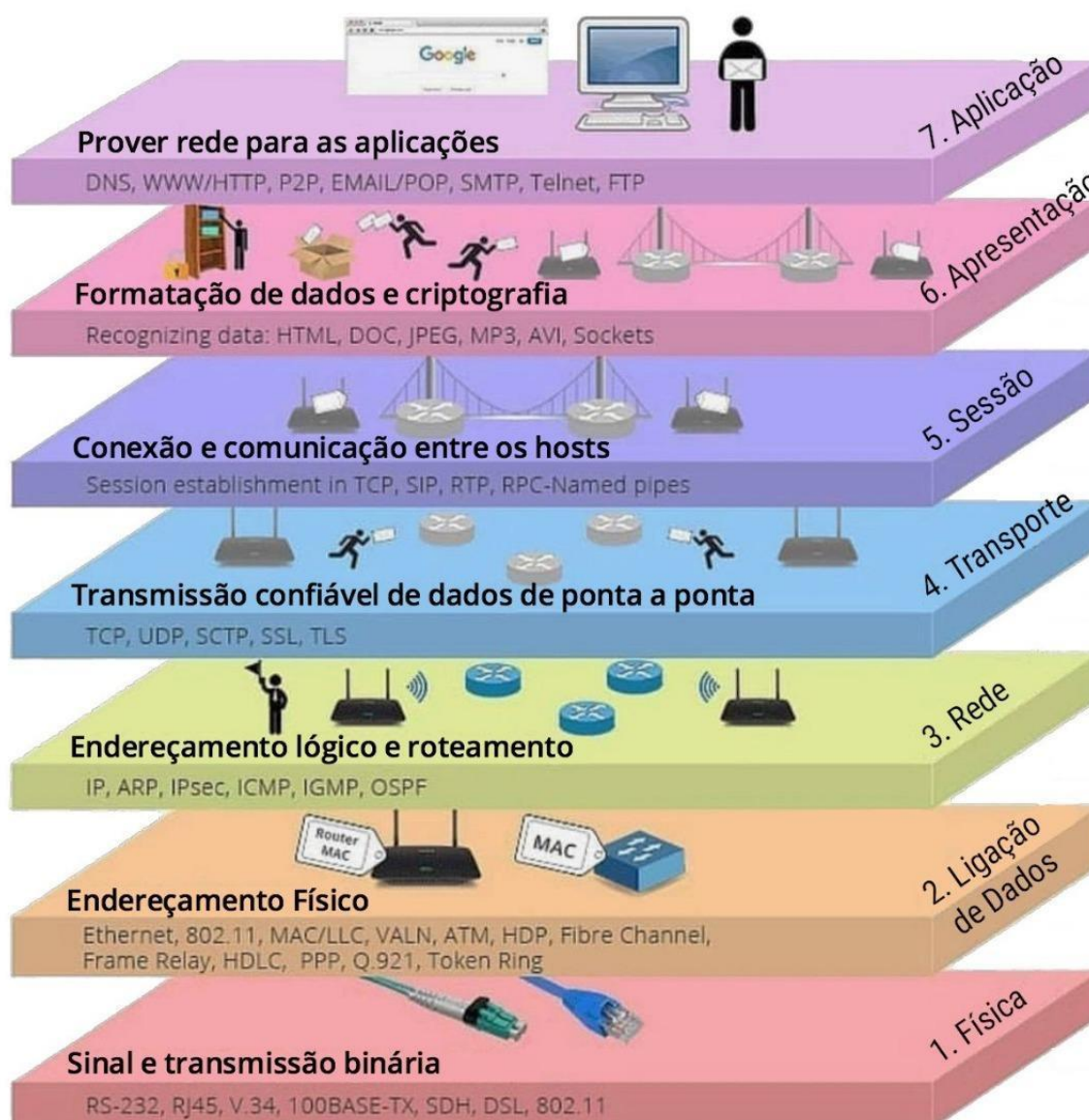
Camada de Transporte: Garante a comunicação fim a fim, segmentando os dados da camada de sessão para transmissão (camada 3) e remontando-os no receptor para entrega à camada superior. (FOROUZAN; FEGAN; GRIESI, 2008)

Camada de Sessão: Controla o estabelecimento, manutenção e término de sessões entre dispositivos. Assegura que a comunicação permaneça ativa durante todo o processo de troca de dados.

Camada de Apresentação: Prepara os dados para a aplicação, realizando tradução de formatos, criptografia e compactação. Essa camada adapta mensagens para garantir compatibilidade entre sistemas com métodos distintos de codificação. (FOROUZAN; FEGAN; GRIESI, 2008)

Camada de Aplicação: Única camada que interage diretamente com o usuário, fornecendo serviços como correio eletrônico e navegação *web*. Gerência protocolos e manipulação de dados conforme a interação do usuário. A Figura abaixo demonstra as camadas do modelo OSI e alguns protocolos.

Figura 1 – Modelo OSI e suas camadas.



Fonte: Adaptada de HUARANCA, 2023

2.4 - CLASSIFICAÇÃO DAS REDES DE COMPUTADORES

As redes de computadores podem ser classificadas de acordo com sua abrangência e finalidade. As Redes Locais (LAN) são projetadas para ambientes de pequeno porte, como residências e escritórios, utilizando tecnologias consolidadas como *Ethernet* para conexões cabeadas e *Wi-Fi* para acesso sem fio. Um desenvolvimento importante no âmbito das LANs são as VLANs, que permitem criar redes virtuais independentes em uma mesma infraestrutura física, melhorando a organização e segurança do tráfego. (CLOUDFLARE, [s.d.])

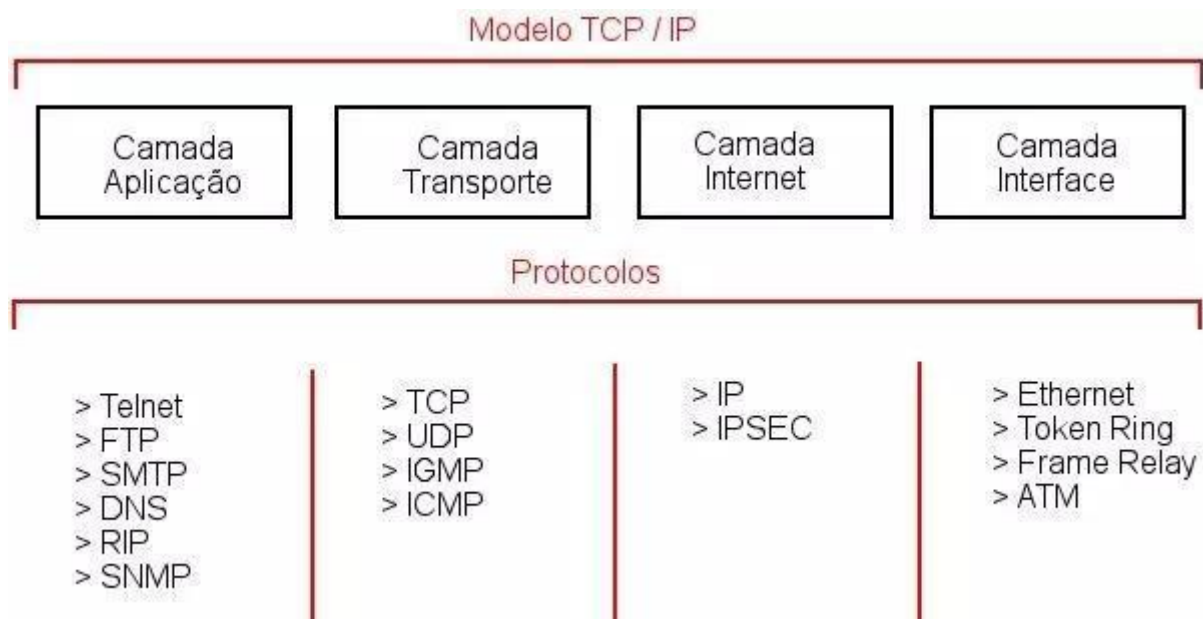
Para áreas maiores, como cidades ou regiões metropolitanas, existem as Redes Metropolitanas (MAN), que normalmente empregam fibra óptica como meio de transmissão principal, oferecendo alta capacidade para conectar múltiplas LANs. Em escala ainda maior, as Redes de Longa Distância (WAN) interligam redes geograficamente distribuídas, com a internet sendo o exemplo mais abrangente deste tipo. (CLOUDFLARE, [s.d.])

No âmbito pessoal, as Redes de Área Pessoal (PAN) facilitam a conexão entre dispositivos próximos, como *smartphones*, *tablets* e periféricos. Sua versão sem fio, a WPAN, utiliza tecnologias como *Bluetooth* para comunicação entre dispositivos em curtas distâncias. Cada uma dessas tipologias apresenta características distintas em termos de alcance, velocidade e aplicações, sendo selecionadas conforme as necessidades específicas de cada cenário de implantação. (CLOUDFLARE, [s.d.])

3 - PROTOCOLO TCP/IP

Em analogia ao sistema nervoso central, que coordena e controla as funções do corpo humano, uma rede de comunicação de dados necessita de um controlador da transferência de dados entre dispositivos IoT. O TCP/IP age da mesma forma, onde o endereço IP seria o identificador do dispositivo que possui acesso à internet, e o TCP o protocolo que permite a troca de dados. O modelo TCP/IP foi criado pela *Defense Advanced Research Projects Agency* (DARPA), no começo da década, em 1969. Ele fez parte do projeto ARPANET, também conhecido como precursor da *Internet*. O objetivo principal era criar uma tecnologia que pudesse transmitir grandes volumes de dados em alta velocidade e resistir a falhas em caso de ataque inimigo, devido ao período de guerra que os Estados Unidos enfrentavam. A Figura 2 representa as camadas do modelo TCP/IP e alguns de seus protocolos. (LIANG, 2025)

Figura 2 – Modelo TCP/IP



Fonte: ATILIO, 2013

3.1 - CAMADAS DO TCP/IP

O protocolo é composto por quatro camadas em hierárquicas: 1) camada de acesso à rede (ou Interface de Rede), 2) camada de *Internet* (ou Rede), 3) camada de Transporte, 4) camada de Aplicação. Essas camadas também possuem seus protocolos internos.

1) A camada de acesso à rede é responsável pela comunicação física entre o computador e a rede. Ela envia de fato o seu pacote ao destinatário, definindo como os dados serão transmitidos, por uma conexão *ethernet* (conexão cabeada), rede sem fio ou *Bluetooth*.

Alguns protocolos desta camada são: *Ethernet*, *Frame Relay*, *Token Ring*, *ATM*.

2) Camada de Rede lida com a transformação dos pacotes de dados em datagramas, onde cada datagrama possui um cabeçalho(*header*), contendo o endereço IP de origem e destino e outros dados relevantes, e a carga útil(*payload*), que contém os dados que serão enviados.

Outros protocolos utilizados por esta camada são: IPv4, IPv6, IPsec.

3) Camada de Transporte fornece a comunicação de ponta-a-ponta entre os sistemas, a principal função seria a confiabilidade e integridade, garantindo a entrega confiável

dos dados, a ordem que chegaram, detectar erros no envio e gerenciar o fluxo de dados.

Os protocolos utilizados nesta camada são: TCP, UDP, IGMP, ICMP.

4) Camada de Aplicação está responsável por fornecer serviços de rede aos aplicativos que estão em execução no dispositivo. Ela interage com os programas do computador e a rede.

Essa é a camada que mais possui protocolos, sendo alguns deles: HTTP, HTTPS, FTP, SFTP, SMTP, IMAP, POP3, DNS, TELNET, SSH, SNMP, NTP, DHCP.

3.2 - PROTOCOLO IP (IPV4 E IPV6)

Protocolo é um padrão para realizar determinadas ações e formatar dados para que os dispositivos se entendam. O Protocolo de *Internet* (IP) é um conjunto de regras para endereçamento e roteamento de pacotes de dados que navegam por roteadores, fazendo com que cheguem ao destino correto. Os dados que trafegam pela *Internet* são destrinchados e são chamados de pacotes. Cada dispositivo que se conecta à *Internet* recebe um Endereço de IP, que conforme os pacotes são enviados para o endereço de IP anexado. Um endereço de IP seria um identificador exclusivo para um dispositivo conectado à *Internet*. Cada IP é uma série de caracteres, havendo diferenças entre IPv4 e IPv6, na qual veremos mais a seguir. Ele identifica exclusivamente um *host* (computador ou outro dispositivo, como uma impressora ou roteador) em uma rede TCP/IP. Para cada dispositivo identificar o IP, utilizamos o *Domain Name System* (DNS). Quando acessamos um domínio, por exemplo, www.unip.br, nosso navegador encontrará o endereço IP do servidor deste domínio. (LIANG, 2025)

3.2 - IPV4 X IPV6

O protocolo IPv4 foi introduzido em 1983 e, apesar de já existir o IPv6, ele ainda é amplamente utilizado atualmente. Seu formato é composto por quatro grupos de números decimais, separados por pontos, totalizando 32 bits. Cada grupo pode variar entre 0 e 255, representando 8 bits do endereço IP. Para que o protocolo TCP/IP funcione corretamente, é necessário o uso das máscaras de sub-rede. Elas permitem identificar se um determinado dispositivo (*host*) pertence à mesma sub-rede ou está localizado em outra rede. (AMAZON, 2023)

A distinção entre a parte do endereço IP que representa a rede e a que representa o host não é fixa. Para fazer essa separação, é necessário um valor adicional — a máscara de sub-rede — também com 32 bits. Sem essa informação, não é possível determinar com clareza os limites entre rede e host. A distribuição dos endereços IP é feita pela InterNIC (Centro de Informações de Rede da Internet), e esses endereços são agrupados em classes. As mais utilizadas são as classes A, B e C. (AMAZON, 2023)

- Classe A utiliza uma máscara de sub-rede padrão de 255.0.0.0, tendo 0-127 o primeiro octeto, por exemplo: 10.52.36.11, onde o primeiro octeto é 10.
- Classe B usam a máscara de sub-rede padrão de 255.255.0.0, sendo o primeiro octeto 128-191, por exemplo: 172.16.52.63, onde seu primeiro octeto é 172.
- Classe C utilizam a máscara de sub-rede padrão de 255.255.255.0, onde o primeiro octeto está entre 192-223, por exemplo: 192.168.123.132, onde o primeiro octeto é 192.
- Classe D, conforme já dito, são utilizados para *multicast*.
- Classe E não está disponível para uso geral, sendo reservado para efeito futuro.

3.3 - ENDEREÇOS PÚBLICOS E PRIVADOS

O endereço IP Privado é o endereço utilizado dentro de uma mesma rede, fazendo com que informações sejam enviadas ou recebidas dentro da mesma rede. O protocolo DHCP, que gerencia uma faixa fixa de endereços, atribui esses dispositivos que se conectam. Nesta rede local, é possível definir um endereço privado fixo, para que caso o dispositivo que faça a distribuição dos endereços desligue, reinicie ou seja substituído, permaneça o mesmo. Também é possível localizar outros dispositivos que estão na mesma rede local e que estão utilizando a mesma faixa de endereço. Já o IP Público seria um sistema utilizado para comunicação com a Internet, seu endereço de IP é atribuído pelo ISP (*Internet Service Provider*). (GEEKSFORGEEKS, 2025; EQUIPE IPSENSE KANDGIGER, 2019)

Endereço de IP Dinâmico: Endereços que se alteram com o tempo, podendo mudar a cada conexão, com o reinício do roteador ou em um período de tempo definido pelo ISP.

Endereço de IP Estático: São endereços fixos que não se alteram com o tempo, até que seja forçadamente alterado.

É possível localizar outros IPs públicos, porém, é possível obter apenas algumas informações, como provedor de *internet* e localização geográfica geral. (FARRIER, 2021)

3.4 - NAT (NETWORK ADDRESS TRANSLATION) E ESCASSEZ DE ENDEREÇOS IP

Os endereços IP estão cada vez mais escassos devido ao número limitado de endereços disponíveis em seu protocolo de 32 bits e o crescimento exponencial da internet com a quantidade de dispositivos que se conectam à *Internet*.

Um ISP poderia ter endereço /16, disponibilizando 65.534 hosts. Caso haja um número de clientes maior que este haveria problemas. Em clientes individuais, podem ser atribuídos os IPs dinâmicos, fornecendo um endereço IP ao se conectar, e obtendo de volta ao desligar a conexão, assim, um único endereço poderia manter até 65.534 usuários ativos. Porém, essa estratégia só funciona para ISPs que possuem mais clientes domésticos que empresas. As empresas têm preferido optar pelo IP estático, por diversos motivos, isso então faz com que todos os clientes comerciais combinados não possam ultrapassar o número de endereços IPs disponíveis que o ISP possui. (Tanenbaum & Wetherall, 2011; AMAZON, 2023)

Com isso, uma das estratégias nascidas foi a forma da NAT (*Network Address Translation*), permitindo que diversos dispositivos compartilhem um único endereço IP público (ou um número pequeno deles). Internamente, a empresa continua recebendo seu endereço IP privado, a diferença ocorre quando um pacote de dados sai da empresa, ele é interceptado e enviado para o ISP que substitui o endereço IP de origem por seu próprio endereço IP público. Após isso, o roteador envia os dados para o dispositivo de destino. Quando o dispositivo de destino retorna com os dados, o roteador intercepta novamente os dados e substitui o endereço IP público pelo endereço IP de origem original, e envia os dados para o dispositivo de origem original. Esse processo é feito, pois a maioria dos pacotes IP transporta uma carga TCP ou UDP, onde ambos possuem um cabeçalho contendo uma porta de origem e uma porta de destino. As portas são inteiras de 16 bits que indicam onde a conexão começa e

termina. Essas portas fornecem o campo necessário para fazer a NAT funcionar. (Tanenbaum & Wetherall, 2011; AMAZON, 2023)

3.5 - IPV6

Quando foi visto que haveria o fim dos endereços IPv4 devido ao crescimento exponencial da internet nos anos 1990, a IETF (*Internet Engineering Task Force*) desenvolveu o IPv6 como solução definitiva, trazendo significativas melhorias em relação ao seu predecessor. O novo protocolo expandiu radicalmente o espaço de endereçamento, saltando de 32 para 128 bits (equivalente a $3,4 \times 10^{38}$ endereços), resolvendo permanentemente o problema de escassez. Além da capacidade quase ilimitada, o IPv6 trouxe otimizações como tabelas de roteamento mais eficientes e compactas, simplificação do cabeçalho para melhor desempenho no processamento, e segurança integrada através do IPsec nativo. Outros avanços incluem suporte aprimorado a QoS (*Quality of Service*) para priorização de tráfego, mecanismos robustos de *multicast* com definição clara de escopos, e auto configuração simplificada de redes (*plug-and-play*). O protocolo também foi projetado para coexistir harmonicamente com o IPv4 durante períodos de transição, enquanto oferece maior escalabilidade e flexibilidade nas políticas de roteamento. Sua estrutura hexadecimal de 8 grupos de 16 bits (ex: 2001:0db8:85a3::8a2e:0370:7334) suporta três tipos de endereçamento: *unicast* (comunicação ponto-a-ponto), *multicast* (transmissão para grupos) e *anycast* (entrega ao nó mais próximo de um grupo), consolidando-se como a base tecnológica para a internet do futuro, mais segura, eficiente e preparada para a explosão de dispositivos conectados. A Figura 3 ilustra as diferenças entre o IPv4 e o IPv6. (DANTE NETWORK, 2021)

Figura 3 – IPv4 x IPv6

ITEM	IPv4	IPv6
Comprimento	32 bits	128 bits
Quantidade de endereços	$4,29 \times 10^9$	$3,4 \times 10^{38}$
Formato do endereço	Decimal	Hexadecimal
Tamanho do cabeçalho	20-60 bytes	Fixo em 40 bytes
Integridade ponta-a-ponta	✗	✓
Segurança	✓ depende da aplicação	✓ IPSec nativo
Criptografia e Autenticação	✗	✓
Possui Checksum	✓	✗
Esquema de Transmissão	Multicast	Multicast e Anycast
Identificação do fluxo de pacotes	✗	✓ Header Flow Label



Fonte: HALLEN, 2021

4 - SOCKETS DE BERKELEY

4.1 - CONCEITO

Em 1980, a ARPA (*Advanced Research Projects Agency*) órgão do governo norte-americano, forneceu recursos financeiros à Universidade da Califórnia, em Berkeley, com a finalidade de que fosse desenvolvida uma implementação UNIX do pacote de protocolos TCP/IP. E assim foi criado o que se conhece como interface de sockets de Berkeley ou simplesmente *sockets* de Berkeley. Um *socket* é uma interface de *software* que permite que dois processos (programas) troquem dados entre si — seja em um mesmo computador ou em máquinas diferentes, conectadas por uma rede. Em termos simples, um *socket* funciona como uma porta de entrada/saída que conecta dois pontos para comunicação bidirecional. De forma similar, os *sockets* de Berkeley são uma API (*Application Programming Interface*) padronizada para a comunicação entre processos. São comumente utilizados para interação entre cliente e servidor, onde, de forma típica, o sistema coloca o servidor em apenas uma máquina com os clientes em outras máquinas. Os clientes se conectam ao servidor, trocam informações e, em seguida, se desconectam; tudo isso através de redes (como a *Internet*).

4.2 - FUNDAMENTOS DOS SOCKETS

A comunicação via *sockets*, geralmente segue o modelo cliente-servidor, onde o servidor espera por conexões, aceita pedidos, e responde. Essa estrutura é a base para a maioria dos serviços de rede (*web*, *e-mail*, banco de dados, entre outros). (IBM, 2023)

4.2.1 - ENDEREÇAMENTO

Para que haja comunicação em rede através de *sockets*, é necessário que se tenha o endereço IP do computador (para identificá-lo na rede) e o número de porta do aplicativo ao qual se quer realizar a conexão (para identificar o processo dentro do dispositivo). É necessário também que se tenha um protocolo. O protocolo TCP/IP oferece dois modos de utilização de *sockets*: o modo orientado a conexão, que funciona sobre o protocolo TCP e o modo orientado a datagrama, que funciona sobre o protocolo UDP, que se localiza na camada de transporte do protocolo TCP/IP. (IBM, 2023)

4.3 - TIPOS DE SOCKETS

- SOCK_STREAM (SOCKETS DE FLUXO):

Os *sockets* de fluxo (*socket stream*) fornecem fluxos de bytes bidirecionais sequenciados com um mecanismo de transmissão para dados de fluxo. Este tipo de soquete transmite dados de forma confiável, em ordem e com recursos fora de banda. No domínio UNIX, o tipo de soquete SOCK_STREAM funciona como um *pipe*. No domínio da *Internet*, o tipo de soquete SOCK_STREAM é implementado no protocolo *Transmission Control Protocol/Internet Protocol* (TCP/IP). Sua utilização é ideal para navegação *web* (HTTP/HTTPS), aplicações que exigem confiabilidade, como *login* em *sites*, banco de dados, envio de arquivos, entre outros. Também é comumente utilizado em aplicações de comunicação de longo prazo, como *chats* e jogos *online*. (IBM, 2023)

- SOCK_DGRAM (SOCKETS DE DATAGRAMA):

Os *sockets* de datagrama, fornecem datagramas, que são mensagens sem conexão de comprimento máximo fixo. Esse tipo de soquete geralmente é usado para mensagens curtas, como um servidor de nomes ou de tempo, porque a ordem e a confiabilidade da entrega da mensagem não são

garantidas. Neste caso, cabe ao programador a detecção de perda de pacotes e reordenação e verificação de integridade, se necessário. No domínio UNIX, o tipo de soquete SOCK_DGRAM é semelhante a uma fila de mensagens. No domínio da *Internet*, o tipo de soquete SOCK_DGRAM é implementado no protocolo UDP/IP (*User Datagram Protocol/Internet Protocol*). Um soquete de datagrama suporta o fluxo bidirecional de dados, que não é sequenciado, confiável ou não duplicado. Um processo que recebe mensagens em um soquete de datagrama pode encontrar mensagens duplicadas ou em uma ordem diferente da ordem enviada. No entanto, os limites de registro nos dados são preservados. Sua utilização é ideal para aplicações em tempo real onde a velocidade importa mais que a confiabilidade, como no caso de *streamings* de áudio/vídeo, jogos *online*, DNS, DHCP, VoIP. (IBM, 2023).

- SOCK_RAW (SOCKETS BRUTOS):

Os *sockets raw* (sockets brutos) fornecem acesso a protocolos e interfaces de rede interna. Permitem que o programador possa criar, modificar, enviar ou capturar pacotes IP manualmente, incluindo cabeçalhos e *payload*. Em outras palavras, o programador possui acesso total. Isso ocorre porque o *raw socket* é um tipo de *socket* que ignora parte da pilha TCP/IP tradicional, permitindo ao usuário manipular diretamente os protocolos de baixo nível, como IP, ICMP, ou protocolos personalizados. No entanto, em sistemas Unix/Linux, só processos com privilégios de *root* podem criar *raw sockets*. Os soquetes brutos são destinados a usuários avançados que desejam aproveitar algum recurso de protocolo que não é diretamente acessível por meio de uma interface normal, ou que desejam criar novos protocolos sobre protocolos de baixo nível existentes. Os *sockets raw* normalmente são orientados a datagramas, embora suas características exatas dependam da interface fornecida pelo protocolo. São utilizados para realizar diagnóstico e teste de redes, para a criação de protocolos customizados (quando você quer desenvolver seu próprio protocolo de rede, fora do TCP/UDP), segurança e análise de intrusão e na manipulação especializada de pacotes. (IBM, 2023).

4.4 - GERENCIAMENTO DE MÚLTIPLAS CONEXÕES

Quando se há a necessidade de um servidor atender vários clientes ao mesmo tempo, ele precisa de uma forma de não bloquear completamente ao atender um único cliente, ou seja, precisa lidar com múltiplas conexões simultâneas. Para que isso seja possível, algumas técnicas são adotadas, como o uso de *Threads*, visto como um modelo tradicional; a técnica de multiplexação com *select()*, *poll()* ou *epoll()*; e assíncrono / não-bloqueante (*async I/O*).

4.5 - THREADS OU PROCESSOS

Uma das formas de lidar com múltiplas conexões simultâneas é com o uso de *Threads*. Cada vez que o servidor aceita uma conexão, ele cria um novo *thread* para se conectar com o cliente. Dessa forma, o servidor continua aceitando novas conexões enquanto trata as anteriores em paralelo. É uma abordagem vantajosa quando se trata de aplicações com poucos clientes, além de ser fácil de entender e implementar. No entanto, os *threads* consomem muita memória, portanto, possui uma escalabilidade limitada de clientes.

No caso dos processos, seguem o mesmo conceito dos *threads*, mas o detalhe é que o servidor cria um novo processo para cada cliente, ou seja, cada cliente é atendido por um processo independente.

4.6 - MULTIPLEXAÇÃO COM SELECT(), POLL() OU EPOLL()

Esse modelo utiliza um único processo e um único *thread*, mas consegue atender vários clientes ao mesmo tempo de forma não bloqueante.

4.6.1 - SELECT()

Trata-se de uma função da biblioteca de *sockets* que monitora vários descritores de arquivo (*sockets*). O servidor agrupa todos os *sockets* ativos em um conjunto chamado *select()*. Quando um cliente envia dados, o *select()* retorna, e o servidor processa a informação. É mais leve e escalável que o modelo por *threads*/processos, devido que não cria múltiplas *threads*. No entanto, tem uma limitação no número de *sockets* que podem ser utilizados, geralmente 1024, além de apresentar baixa performance com muitos clientes.

4.6.2 - POLL()

É uma versão mais moderna do *select()*, possuindo a mesma lógica, mas não tendo limite de *sockets* a serem utilizados. Possui boa performance com grande número de clientes.

4.6.3- EPOLL()

Ideal quando se tem uma aplicação para diversas conexões. Usa um modelo orientado a eventos. É eficiente, pois o kernel informa quando um *socket* está pronto.

4.7 - ASSÍNCRONO / NÃO BLOQUEANTE (ASYNC I/O)

O modelo é totalmente não-bloqueante quando implementado em linguagens como Node.js, Python (*asyncio*), Go, ou Rust *async*. Usa “*event loop*” e *callbacks* ou corrotinas. O servidor está sempre disponível, apesar do código parecer linear.

5 - PLANO DE DESENVOLVIMENTO DA APLICAÇÃO

Este projeto consiste na implementação de um servidor de chat seguro, desenvolvido em Python, com o objetivo de permitir a comunicação criptografada entre múltiplos clientes. A solução combina três tecnologias principais: *sockets* para comunicação em rede, *Fernet* para criptografia de mensagens e *tkinter* para a interface gráfica. A arquitetura adotada garante segurança na troca de mensagens, aliada a uma interface intuitiva para os usuários.

O sistema é estruturado em torno da classe `chatServer`, responsável por gerenciar as operações do servidor, incluindo conexões de clientes, envio e recebimento de mensagens. As principais funcionalidades implementadas são:

- `ligarServidor()`: Inicializa o servidor, configurando os parâmetros de rede e permitindo a conexão simultânea de múltiplos clientes.
- `aceitarConexoes()`: Fica em estado de espera, aceitando novas conexões de clientes e registrando-os em uma lista de conexões ativas.
- `gerenciarCliente(conn, addr)`: Mantém a comunicação individual com cada cliente conectado, utilizando *threads* dedicadas para garantir concorrência.
- `enviarMensagem(event=None)`: Envia mensagens criptografadas ao cliente selecionado, assegurando a confidencialidade dos dados.
- `fechar()`: Encerra todas as conexões ativas de maneira controlada, finalizando o servidor com segurança.

Adicionalmente, foi desenvolvida uma interface gráfica utilizando *tkinter*, que permite a seleção de clientes conectados e o envio de mensagens de forma intuitiva, facilitando a interação do usuário com o sistema.

- *Sockets*: Optou-se pela utilização de *sockets* devido à sua natureza multiplataforma e eficiência na comunicação entre dispositivos, dispensando a necessidade de *frameworks* externos.
- *Fernet*: A criptografia das mensagens foi implementada utilizando a biblioteca *Fernet*, que oferece um equilíbrio entre simplicidade e robustez. Baseado no algoritmo AES-128, garante que as mensagens não sejam interceptadas ou decodificadas por terceiros, atendendo aos requisitos de segurança do projeto.
- *Tkinter*: A interface gráfica foi desenvolvida com *tkinter*, biblioteca nativa do Python, assegurando compatibilidade entre diferentes sistemas operacionais e

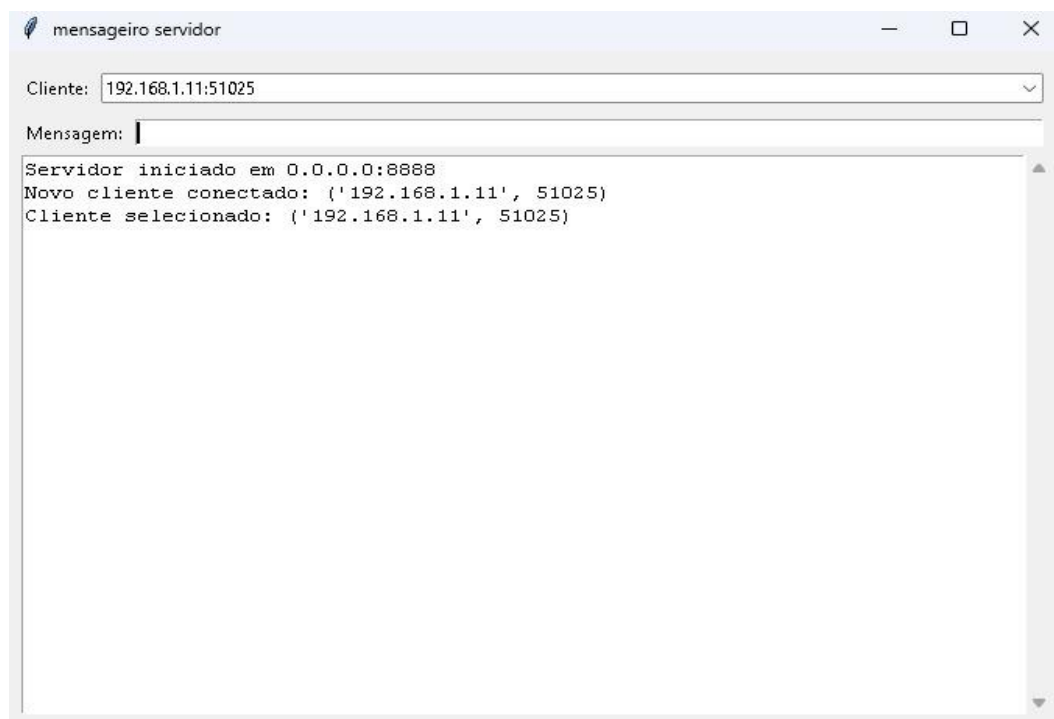
facilitando a manutenção do código. A escolha priorizou a usabilidade e a integração com as demais componentes do sistema.

- *Threading*: Para lidar com múltiplas conexões simultâneas sem comprometer a responsividade da interface, utilizou-se *threading*. Essa abordagem permite que o servidor gerencie cada cliente em um *thread* separado, garantindo concorrência e eficiência no processamento das mensagens.

A implementação proposta demonstra a viabilidade de um sistema de comunicação seguro, combinando técnicas de criptografia, programação concorrente e desenvolvimento de interfaces gráficas. O projeto atende aos requisitos de segurança e usabilidade, servindo como base para futuras expansões, como a inclusão de autenticação de usuários ou suporte a grupos de *chat*.

a Figura 4 demonstra o aplicativo do lado do servidor, permite a seleção de um cliente específico e oferece uma caixa de texto para o envio de mensagens ao usuário.

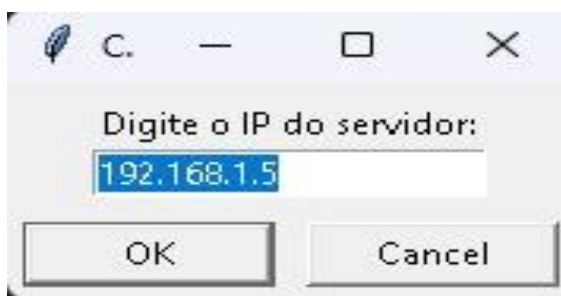
Figura 4 – Interface servidor



Fonte: Autoria Própria, 2025.

A Figura 5 ilustra a interface de conexão utilizada pelo cliente para estabelecer a conexão com o servidor por meio do endereço IP.

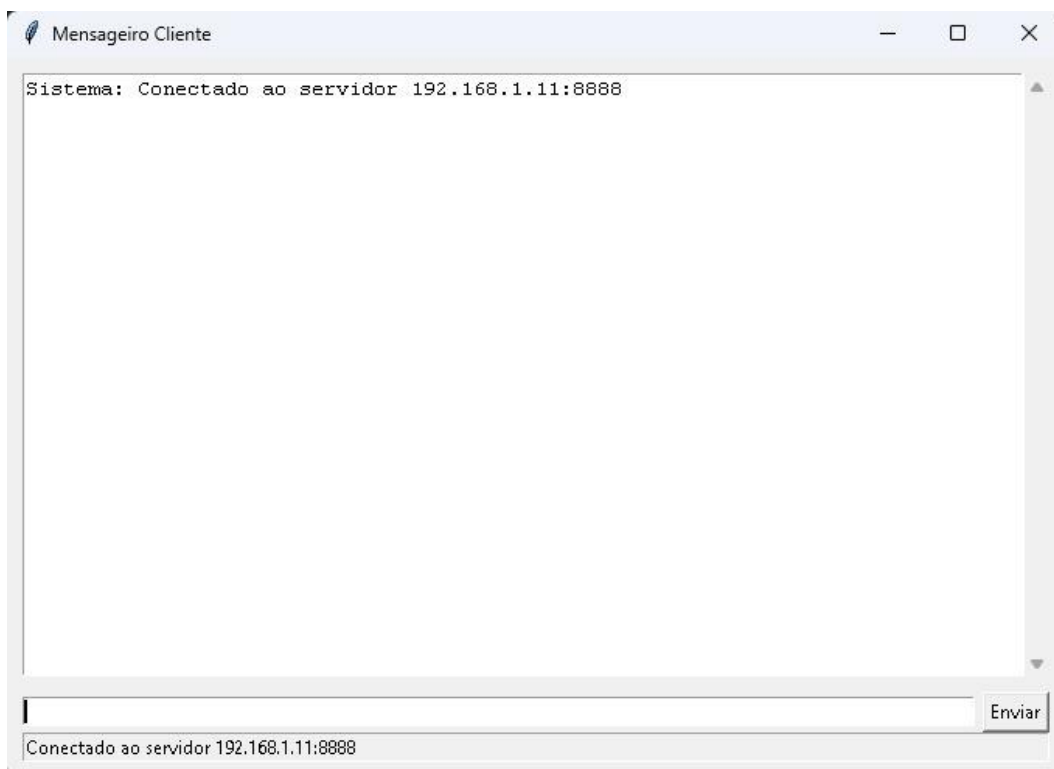
Figura 5 – Interface de conexão



Fonte: Autoria Própria, 2025.

Já a Figura 6 exibe a interface do lado do cliente, na qual é possível receber mensagens enviadas pelo servidor e enviar respostas.

Figura 6 – Interface do cliente



Fonte: Autoria Própria, 2025.

6 - RELATÓRIOS COM LINHAS DE CÓDIGO

6.1 - CÓDIGO DO CLIENTE

```
import socket

from cryptography.fernet import Fernet

import threading

import tkinter as tk

from tkinter import scrolledtext, messagebox, simpledialog

class ChatClientGUI:

    def __init__(self, root):

        self.root = root

        self.root.title("Mensageiro Cliente")

        # Configurações

        self.chave = b'NQzcwdHBP1wAT2YoZLmSwliGV8IztiXUj0Ocu7+7wAl='

        self.fernet = Fernet(self.chave)

        self.PORT = 8888

        self.client_socket = None

        # Solicita o IP do servidor

        self.HOST = self.solicitar_ip()

        if not self.HOST: # Se o usuário cancelar

            self.root.destroy()

            return

        # Configura a interface e conecta ao servidor

        self.configInterface()
```

```

self.cenectServer()

def solicitar_ip(self):

    """Solicita o IP do servidor através de uma caixa de diálogo"""

    self.root.deiconify() # Mostra a janela principal antes do diálogo

    ip = simpledialog.askstring(

        "Configuração de Conexão",

        "Digite o IP do servidor:",

        parent=self.root,

        initialvalue="192.168.1.5" # Valor padrão

    )

    return ip

def configInterface(self):

    """Configura os elementos da interface"""

    main_frame = tk.Frame(self.root, padx=10, pady=10)

    main_frame.pack(fill=tk.BOTH, expand=True)

    # Área de chat

    self.chat_area = scrolledtext.ScrolledText(main_frame, state='disabled')

    self.chat_area.pack(fill=tk.BOTH, expand=True, pady=(0, 10))

    # Entrada de mensagem

    input_frame = tk.Frame(main_frame)

    input_frame.pack(fill=tk.X)

    self.message_entry = tk.Entry(input_frame)

    self.message_entry.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=(0, 5))

```

```

self.message_entry.bind("<Return>", self.enviaMensagens)

        send_btn      =      tk.Button(input_frame,      text="Enviar",
command=self.enviaMensagens)

    send_btn.pack(side=tk.RIGHT)

    # Barra de status

    self.status_var = tk.StringVar()

    self.status_var.set(f"Conectando a {self.HOST}:{self.PORT}...")

        status_bar  =  tk.Label(main_frame, textvariable=self.status_var, bd=1,
relief=tk.SUNKEN, anchor=tk.W)

    status_bar.pack(fill=tk.X)

    # Configuração de fechamento

    self.root.protocol("WM_DELETE_WINDOW", self.fechar)

def cenectServer(self):

    """Conecta ao servidor"""

    try:

        self.client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

        self.client_socket.connect((self.HOST, self.PORT))

        self.status_var.set(f"Conectado ao servidor {self.HOST}:{self.PORT}")

                self.caixaMensagem("Sistema",  f"Conectado  ao  servidor
{self.HOST}:{self.PORT}")

        # Thread para receber mensagens

        threading.Thread(target=self.reberMensagens, daemon=True).start()

```

```

        self.message_entry.focus()

    except Exception as e:

        messagebox.showerror("Erro", f"Não foi possível conectar ao servidor: {e}")

        self.root.destroy()

def caixaMensagem(self, sender, message):

    """Exibe mensagem na área de chat"""

    self.chat_area.config(state='normal')

    self.chat_area.insert(tk.END, f"{sender}: {message}\n")

    self.chat_area.config(state='disabled')

    self.chat_area.see(tk.END)

def reberMensagens(self):

    """Recebe mensagens do servidor"""

    while True:

        try:

            data = self.client_socket.recv(1024)

            if not data:

                break

            message = self.fernet.decrypt(data).decode()

            self.root.after(0, self.caixaMensagem, "Servidor", message)

        except Exception as e:

            self.root.after(0, self.caixaMensagem, "Sistema", f"Conexão encerrada: {e}")

            self.root.after(0, lambda: self.status_var.set("Desconectado"))

            break

```

```

def enviaMensagens(self, event=None):

    """Envia mensagem para o servidor"""

    message = self.message_entry.get().strip()

    if not message:

        return

    try:

        encrypted_msg = self.fernet.encrypt(message.encode())

        self.client_socket.sendall(encrypted_msg)

        self.caixaMensagem("Você", message)

        self.message_entry.delete(0, tk.END)

    except Exception as e:

        self.caixaMensagem("Sistema", f"Erro ao enviar mensagem: {e}")

        self.status_var.set("Desconectado")

def fechar(self):

    """Fecha a conexão ao sair"""

    try:

        if self.client_socket:

            encrypted_msg = self.fernet.encrypt("tt".encode())

            self.client_socket.sendall(encrypted_msg)

            self.client_socket.close()

    except:

        pass

    finally:

```

```

        self.root.destroy()

if __name__ == "__main__":

    root = tk.Tk()

    root.withdraw() # pede o ip, deixando o mensageiro desabilitado

    app = ChatClientGUI(root)

    root.mainloop()

```

6.2 - CÓDIGO DO SERVIDOR

```

import socket

from cryptography.fernet import Fernet

import threading

import tkinter as tk

from tkinter import scrolledtext, messagebox, ttk

class chatServer:

    def __init__(self, root):

        self.root = root

        self.root.title("mensageiro servidor")

# Configurações

        self.chave = b'NQzcwHBP1wAT2YoZLmSwliGV8lztixUj0Ocu7+7wAl='

        self.fernet = Fernet(self.chave)

        self.HOST = '0.0.0.0'

        self.PORT = 8888

        self.server_socket = None

        self.listaDCliente = []

        self.selected_client = None

```

```

# Inicia interface e servidor

self.configInterface()

self.ligarServidor()

def configInterface(self):

    """Configura a interface gráfica"""

    main_frame = tk.Frame(self.root, padx=10, pady=10)

    main_frame.pack(fill=tk.BOTH, expand=True)

    # menu de clientes

    client_frame = tk.Frame(main_frame)

    client_frame.pack(fill=tk.X, pady=5)

    tk.Label(client_frame, text="Cliente:").pack(side=tk.LEFT)

    self.client_combobox = ttk.Combobox(client_frame, state="readonly")

    self.client_combobox.pack(side=tk.LEFT, padx=5, fill=tk.X, expand=True)

    self.client_combobox.bind("<<ComboboxSelected>>", self.selectCliente)

    # envio de mensagem

    send_frame = tk.Frame(main_frame)

    send_frame.pack(fill=tk.X, pady=5)

    tk.Label(send_frame, text="Mensagem:").pack(side=tk.LEFT)

    self.message_entry = tk.Entry(send_frame)

    self.message_entry.pack(side=tk.LEFT, padx=5, fill=tk.X, expand=True)

    self.message_entry.bind("<Return>", self.enviarMensagem)

    # Área de log

    self.log_area = scrolledtext.ScrolledText(main_frame, state='disabled')

    self.log_area.pack(fill=tk.BOTH, expand=True)

```

```

# fechamento

self.root.protocol("WM_DELETE_WINDOW", self.fechar)

def atualizarListaClientes(self):

    """Atualiza a lista de clientes no combobox"""

    clients = [f'{addr[0]}:{addr[1]}' for conn, addr in self.listaDCliente]

    self.client_combobox['values'] = clients

    if clients and not self.client_combobox.get():

        self.client_combobox.current(0)

        self.selectCliente()

def selectCliente(self, event=None):

    """Atualiza o cliente selecionado"""

    selection = self.client_combobox.get()

    if selection:

        ip, port = selection.split(':')

        for conn, addr in self.listaDCliente:

            if addr[0] == ip and str(addr[1]) == port:

                self.selected_client = (conn, addr)

                self.LogMensagens(f"Cliente selecionado: {addr}")

                self.message_entry.focus()

                break

def LogMensagens(self, message):

    """Adiciona mensagem ao log"""

    self.log_area.config(state='normal')

```

```

self.log_area.insert(tk.END, message + "\n")

self.log_area.config(state='disabled')

def ligarServidor(self):

    """Inicia o servidor automaticamente"""

    try:

        self.server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

        self.server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

        self.server_socket.bind((self.HOST, self.PORT))

        self.server_socket.listen(5)

        self.LogMensagens(f"Servidor iniciado em {self.HOST}:{self.PORT}")

        # Thread para aceitar conexões

        threading.Thread(target=self.aceitarConexoes, daemon=True).start()

    except Exception as e:

        self.LogMensagens(f"Erro ao iniciar servidor: {e}")

        messagebox.showerror("Erro", f"Não foi possível iniciar o servidor: {e}")

        self.root.destroy()

def aceitarConexoes(self):

    """Aceita novas conexões de clientes"""

    while True:

        try:

            conn, addr = self.server_socket.accept()

            self.listaDCliente.append((conn, addr))

```

```

self.LogMensagens(f"Novo cliente conectado: {addr}")

# Atualiza lista de clientes

self.root.after(0, self.atualizarListaClientes)

# Thread para lidar com o cliente

threading.Thread(

    target=self.gerenciarCliente,

    args=(conn, addr),

    daemon=True

).start()

except Exception as e:

    if not isinstance(e, OSError):

        self.LogMensagens(f"Erro ao aceitar conexão: {e}")

    break

def gerenciarCliente(self, conn, addr):

    """Lida com a comunicação de um cliente"""

    while True:

        try:

            data = conn.recv(1024)

            if not data:

                break

            msg = self.fernet.decrypt(data).decode()

            if msg == "tt":

                break

            self.LogMensagens(f"{addr} disse: {msg}")

```

```

except Exception as e:

    self.LogMensagens(f"Erro com {addr}: {e}")

    break

conn.close()

if (conn, addr) in self.listaDCliente:

    self.listaDCliente.remove((conn, addr))

    self.LogMensagens(f"{addr} desconectou")

    self.root.after(0, self.atualizarListaClientes)

def enviarMensagem(self, event=None):

    """Envia mensagem para o cliente selecionado"""

    if not self.selected_client:

        messagebox.showwarning("Aviso", "Nenhum cliente selecionado!")

        return

    message = self.message_entry.get()

    if not message:

        return

    conn, addr = self.selected_client

    try:

        encrypted_msg = self.fernet.encrypt(message.encode())

        conn.sendall(encrypted_msg)

        self.LogMensagens(f"Você para {addr}: {message}")

        self.message_entry.delete(0, tk.END)

    except Exception as e:

        self.LogMensagens(f"Erro ao enviar mensagem: {e}")

```

```

        if (conn, addr) in self.listaDCliente:

            self.listaDCliente.remove((conn, addr))

            self.root.after(0, self.atualizarListaClientes)

def fechar(self):

    """Fecha o servidor corretamente"""

    try:

        for conn, addr in self.listaDCliente:

            try:

                conn.close()

            except:

                pass

        if self.server_socket:

            self.server_socket.close()

            self.root.destroy()

    except Exception as e:

        self.LogMensagens(f"Erro ao fechar servidor: {e}")

        self.root.destroy()

if __name__ == "__main__":

    root = tk.Tk()

    app = chatServer(root)

    root.mainloop()

```

REFERÊNCIAS

AMAZON. **Qual é a diferença entre IPv4 e IPv6?** Disponível em: <https://aws.amazon.com/pt/compare/the-difference-between-ipv4-and-ipv6/>. Acesso em: 12 abr. 2025.

ATILIO, D.. **Entendendo o TCP/IP.** Disponível em: <https://terminaldeinformacao.com/2013/02/28/entendendo-o-tcpip/>. Acesso em: 11 abr. 2025.

CLOUDFLARE. **O que é uma LAN (rede local)?** Disponível em: <https://www.cloudflare.com/pt-br/learning/network-layer/what-is-a-lan/>. Acesso em: 5 abr. 2025.

CLOUDFLARE. **O que é uma rede de área metropolitana (MAN)?** Disponível em: <https://www.cloudflare.com/pt-br/learning/network-layer/what-is-a-metropolitan-area-network/>. Acesso em: 5 abr. 2025.

CLOUDFLARE. **O que é uma rede de área pessoal (PAN)?** Disponível em: <https://www.cloudflare.com/pt-br/learning/network-layer/what-is-a-personal-area-network/>. Acesso em: 5 abr. 2025.

CLOUDFLARE. **O que é uma WAN? | WAN X LAN.** Disponível em: <https://www.cloudflare.com/pt-br/learning/network-layer/what-is-a-wan/>. Acesso em: 5 abr. 2025.

DANTE NETWORK. **Tipos de endereço IPv6.** Disponível em: <https://ccna.network/tipos-de-endereco-ipv6/>. Acesso em: 12 abr. 2025.

EQUIPE IPSENSE KANDGIGER. **Entenda tudo sobre o protocolo DHCP!** Disponível em: <https://www.ipsense.com.br/amazon-web-services/entenda-tudo-sobre-o-protocolo-dhcp/>. Acesso em: 12 abr. 2025.

FARRIER, E.. **Endereços IP públicos x privados: Qual é a diferença?** Disponível em: <https://www.avast.com/pt-br/c-ip-address-public-vs-private>. Acesso em: 12 abr. 2025.

FOROUZAN, B. A.; SOPHIA CHUNG FEGAN; ARIIVALDO GRIESI. **Comunicação de dados e redes de computadores.** São Paulo: McGraw Hill, 2008.

GEEKSFORGEEKS. **Difference between Private and Public IP addresses.** Disponível em: <https://www.geeksforgeeks.org/difference-between-private-and-public-ip-addresses/>. Acesso em: 12 abr. 2025.

HALLEN, E.. **O que é IP? Quais as diferenças entre IPv4 e IPv6?** Disponível em: <https://edca.com.br/blog/o-que-e-ip-quais-as-diferencas-entre-ipv4-e-ipv6>. Acesso em: 12 abr. 2025.

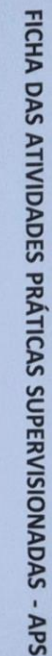
HUARANCA, T.. **Protocolos básicos de rede na compreensão do modelo OSI.** Disponível em: <https://community.cisco.com/t5/artigos-gerais/protocolos-b%C3%AAsicos-de-rede-na-compreens%C3%A3o-do-modelo-osi/ta-p/4802414>. Acesso em: 5 abr. 2025.

IBM. **Como os sockets funcionam.** IBM Documentation, 2023. Disponível em: <https://www.ibm.com/docs/pt-br/i/7.5.0?topic=programming-how-sockets-work>. Acesso em: 13 abr. 2025.

IBM. **Tipos de socket.** IBM Documentation, 2023. Disponível em: <https://www.ibm.com/docs/pl/aix/7.1?topic=protocols-socket-types>. Acesso em: 13 abr. 2025.

LIANG, H.. **Compreender os conceitos básicos de endereçamento TCP/IP e sub-redes.** Disponível em: <https://learn.microsoft.com/pt-br/troubleshoot/windows-client/networking/tcpip-addressing-and-subnetting>. Acesso em: 11 abr. 2025.

TANENBAUM, A. S.; WETHERALL, D. J.. **Redes de computadores.** São Paulo: Pearson Prentice Hall, 2011.



RA: G783HF2

TURNNO: NOTURNOSEMESTRE: 5º SEMESTRE[illegible]

TOTAL DE HORAS ATRIBUIDAS: 80h

AVALIAÇÃO: 4 PROUNDO

NOTA: 9,0 Aprovado ou Reprovado Reprovado

DATA: 24 / 05 / 25

CARIMBO E ASSINATURA DO COORDENADOR DO CURSO

Prof. Dr. Fernando José Alho Gottlieb
Coordenador Auxiliar
Ciência da Computação



RA: N239239

TURNO: NOTURNO

SEMESTRE: 5º SEMESTRE

[illegible]

(1) Horas atribuídas de acordo com o regulamento das Atividades Supervisionadas do curso, a ser avaliada com o supervisor.

TOTAL DE HORAS ATRIBUÍDAS:

409

AValiação:

Approved

Aprovado ou Reprovado

NOTA:

90

DATA: 26/05/2023

Prof. Dr. Fernando José Alho Gotti
Coordenador Auxiliar
da Computação

CARIMBO E ASSINATURA DO COORDENADOR DO CURSO

